



Module 4 : Statistical and Neural NLP Models

Dr. Priya R L, Ms. Sunita Suralkar

Faculty Incharge for NCMPE64 (Natural Language Processing & Generative AI)

Department of Computer Engineering

VES Institute of Technology, Mumbai

Agenda - Part : I

- **Statistical Language Models:**
 - N-grams,
 - Smoothing,
 - Perplexity Neural
- **Networks for NLP:**
 - RNNs,
 - LSTMs
 - GRUs



Language Model

What is a Language Model (LM)? A language model is a machine learning model that predicts upcoming words.

Core Task: Predicting the **next word in a sequence**: $P(w_n \mid w_1, w_2, \dots, w_{n-1})$.

Why is this important? It is the foundation of **speech recognition, machine translation, autocorrect, and modern Generative AI**.

The Problem: Calculating the exact probability of long sequences is impossible due to the infinite number of possible word combinations.

Language Models

- LM tries to understand the "**rules**" of a language by looking at **probabilities**.
- If I say, "The cat sat on the...", you instinctively know the next word is likely "mat" and highly unlikely to be "refrigerator".
- A language model mathematically quantifies this intuition.
- The fundamental challenge in NLP is that language is infinitely creative; we cannot simply build a database of every sentence ever spoken.
- *We need models that can generalize.*

N-Gram Language Model

The Markov Assumption: Instead of looking at the entire past history, we only look at the previous $N-1$ words.

Types of N-grams:

- **Unigram (1-gram):** Words are independent. $P(w_i)$
- **Bigram (2-gram):** Depends on the previous 1 word. $P(w_i | w_{i-1})$
- **Trigram (3-gram):** Depends on the previous 2 words. $P(w_i | w_{i-2}, w_{i-1})$

Calculation: Based on corpus counts (Maximum Likelihood Estimation).

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

Formalizing the N-gram Model

The Goal: Approximate the probability of a word given its entire history:

$$P(w_i | w_1, w_2, \dots, w_{i-1}).$$

The N-gram Approximation: We restrict the history to $N-1$ words.

$$P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-(N-1)}, \dots, w_{i-1})$$

Terminology:

- $N=1$: Unigram. Words are generated independently.
- $N=2$: Bigram. Conditioned on 1 previous word.
- $N=3$: Trigram. Conditioned on 2 previous words.

Start and End Tokens

Problem: How do we calculate the probability of the *first* word if there is no previous word? How does the model know when to stop?

Solution: Introduce special structural tokens.

- **<s>**: Start of sentence token.
- **</s>**: End of sentence token.

Example Sentence: "The cat sat."

- **Padded Sequence:** **<s>** The cat sat **</s>**
- **Bigrams:** (**<s>**, The), (The, cat), (cat, sat), (sat, **</s>**)

Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE): The simplest way to estimate probabilities is by counting frequencies in the corpus and normalizing them.

The Core Intuition: If an event happens frequently in the past, it is highly likely to happen in the future.

Formula for Bigrams:

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}w_i)}{C(w_{i-1})}$$

(Where C represents the raw count of occurrences)

The Core Issue: Data Sparsity

- **The Reality of Text:** Language is a "long-tail" distribution (Zipf's Law). A few words occur constantly (the, a, is), but the vast majority of words occur rarely.
- **The Sparsity Problem:** Even in a corpus of billions of words, most valid N-grams (like "blue refrigerator") will have a count of zero.
- **The MLE Failure:** If $C(w_{i-1}, w_i) = 0$, then Maximum Likelihood Estimation dictates $P(w_i \mid w_{i-1}) = 0$.
- **The Catastrophic Result:** A single zero probability makes the joint probability of the entire sentence exactly zero.

N-Gram Language Model

- N-grams are great, but they suffer from sparsity.
- Even if you train a model on all of Wikipedia, there will be valid two-word combinations it has never seen.
- If a combination has a count of zero, the probability of the entire sentence becomes zero.
- To fix this, we use "Smoothing."

What is Smoothing?

- Smoothing (or discounting) is the process of flattening our probability distributions.
- **The "Robin Hood" Approach:** We steal a small amount of probability mass from the frequent, seen events, and distribute it among the unseen (zero-count) events.
- **The Goal:** Ensure that no valid sequence ever receives an absolute zero probability, while maintaining the relative frequency of seen words.

Laplace (Add-1) Smoothing

Pretend we have seen every possible N-gram at least exactly one more time than we actually did.

$$P_{Laplace}(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + 1}{C(w_{i-1}) + V}$$

The Formula: Add 1 to the numerator, and add the entire Vocabulary size V to the denominator to keep the probabilities summing to 1.

Example: Laplace Smoothing

Scenario: We want $P(\text{"refrigerator"} | \text{"blue"})$.

Corpus Stats: * $C(\text{"blue"}) = 100$

- $C(\text{"blue refrigerator"}) = 0$
- Vocabulary Size (V) = 10,000

MLE Calculation: $0/100 = 0$

Laplace Calculation:

$$P_{Laplace} = \frac{0 + 1}{100 + 10,000} = \frac{1}{10,100} \approx 0.000099$$

Result: It is no longer zero!

Unigram Smoothing: Example

$$P_{LP}(w_i) = \frac{c_i + 1}{N + V}$$

- Tiny Corpus, $V=4$; $N=20$

Word	True Ct	Unigram Prob	New Ct	Adjusted Prob.
eat	10	.5	11	.46
British	4	.2	5	.21
food	6	.3	7	.29
happily	0	.0	1	.04
	20	1.0	~20	1.0

Add-One Smoothing: Bi-gram Effects

$$P_{+1}(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + v}$$

Using $v = 86,700$ compute some example probabilities:

$C(w_{i-1}) = 10,000$		
$C(w_{i-1}, w_i)$	$P_{ML} =$	$P_{+1} \approx$
100	1/100	1/970
10	1/1k	1/10k
1	1/10k	1/48k
0	0	1/97k

$C(w_{i-1}) = 100$		
$C(w_{i-1}, w_i)$	$P_{ML} =$	$P_{+1} \approx$
100	1	1/870
10	1/10	1/9k
1	1/100	1/43k
0	0	1/87k

The Problem with Add-1 Smoothing

- Laplace smoothing steals *too much* probability mass from frequent words.
- In NLP, the vocabulary V is huge (often 50,000+ words). Adding V to the denominator drastically shrinks the probability of very common, highly reliable N-grams.
- Add-1 is a great tool but is rarely used in modern, large-scale NLP systems because it shifts the distribution too aggressively.

Add-k (Fractional) Smoothing

- Instead of adding 1, we add a fractional value k (e.g., $k = 0.5$, $k = 0.01$)

$$P_{Add-k}(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) + k}{C(w_{i-1}) + k \cdot V}$$

- This still guarantees that zero-count items get some probability mass, but it prevents the massive vocabulary size from overwhelming the denominator.
- **Hyperparameter Tuning:** k is chosen by testing different values on a separate validation dataset to see which yields the lowest perplexity.

How Do We Know a Model is "Good"?

- How do we objectively evaluate if one model is better than another?
- We have built a Bigram model with Laplace smoothing and a Trigram model with smoothing. Which one is better?
- **Two Evaluation Paradigms:**
 - **Extrinsic Evaluation:** Testing the model in a real-world application.
 - **Intrinsic Evaluation:** Testing the model purely on its mathematical ability to predict text.
- Never evaluate a model on the data it was trained on.

Extrinsic Evaluation (In Vivo)

- Embed the language model into a downstream NLP task (e.g., Speech Recognition, Machine Translation, or Spell Checking).
- **The Metric:** Does the application perform better? (e.g., Does the speech recognizer make fewer transcription errors?).
- **Pros:** Measures the actual business value and real-world impact of the model.
- **Cons:** Extremely slow, computationally expensive, and can be influenced by other components in the system.

Intrinsic Evaluation (In Vitro)

- Evaluate the model in isolation using a dedicated, unseen **Test Corpus**.
- **The Core Idea:** A good model should not be "surprised" by normal human language.
- **The Metric:** If we feed the model a real, human-written text that it has never seen before, a good model will assign that text a *high* probability.
- **Efficiency:** Fast, mathematically rigorous, and allows for rapid model comparison.

Test Set

- Because extrinsic testing is so slow, we rely heavily on Intrinsic Evaluation.
- We take a chunk of text that the model has never seen—called a test set—and we simply ask the model, "What is the probability that this text exists?"
- If our model has truly learned the underlying rules of English, it should look at this new English text and assign it a very high probability.
- If it assigns it a low probability, the model is "surprised" by normal language, which means it's a poor model.

The Roots of Evaluation: Information Theory

Claude Shannon (1948): The father of Information Theory.

Information as Surprise: The amount of "information" in a message is proportional to how surprising it is.

Example:

- "The sun rose in the **east**." (Highly predictable = Low information/surprise)
- "The sun rose in the **west**." (Highly unpredictable = High information/surprise)

The Goal of an LM: **To minimize surprise when reading valid text.**

A perfect language model should almost never be surprised when reading a well-written book.

Entropy (H)

Definition: Entropy is the expected value of the information of a distribution. It is the theoretical lower bound on the number of bits needed to encode a sequence of data.

Shannon formalized surprise into a metric called Entropy, You can think of entropy as a measure of chaos or uncertainty.

Formula for a source

$$H(X) = - \sum_{x \in X} p(x) \log_2 p(x)$$

Interpretation: High entropy means a language is highly unpredictable (chaotic). Low entropy means it is highly structured.

Cross-Entropy

We don't know the *true* probability distribution of human language (p).
We only have our model's *estimated* distribution (m).

Cross-Entropy Definition: Measures how many bits are needed to encode data from the true distribution p using our flawed model m .

Approximation for a long sequence (W) of length N :

$$H(W) = -\frac{1}{N} \log_2 P(w_1 w_2 \dots w_N)$$

Minimize Cross-Entropy. The closer our model is to the true language, the lower the cross-entropy.

Evaluating Language Models - Perplexity

- **Intrinsic Evaluation:** How well does the model predict a hidden test set?
- **Perplexity (PP):** The **standard evaluation metric** for language models in NLP.
- **Perplexity is exactly $2^{\text{Cross-Entropy}}$**
- **Rule of Thumb:** **Lower perplexity = Better model.**
- **Formula:** The inverse probability of the test set, normalized by the number of words (N).

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}$$

- **Intuition:** Perplexity represents the weighted average branching factor of a language. A perplexity of 100 means the model is, on average, choosing between 100 equally likely words at each step.

The Intuition Behind Perplexity

- **What does a Perplexity of 100 mean?**
- **The Weighted Branching Factor:** It means that, on average, at every single word prediction, the model is as confused as if it had to guess evenly between 100 equally likely words.
- **Analogy:** Rolling a weighted die.
 - Perplexity of 6 = Rolling a 6-sided die for the next word.
 - Perplexity of 10,000 = Rolling a 10,000-sided die.

Evaluating Language Models - Perplexity

- How do we know if our Bigram model is better than our Trigram model?
- Think of perplexity as a measure of the model's "surprise" or "confusion."
- If a model assigns a high probability to the test data, it is not surprised by it, and its perplexity will be low.
- If your model has a perplexity of 50, it means that at every step of generating a sentence, it feels like it is rolling a 50-sided die to pick the next word.



The Data Split: Train, Validate, Test

To ensure valid evaluation, data is split into three buckets:

1. **Training Set (~80%):** Used to calculate the N-gram counts and probabilities.
2. **Validation/Development Set (~10%):** Used to tune hyperparameters (like the k in Add- k smoothing or lambdas in Interpolation) by monitoring perplexity.
3. **Test Set (~10%):** Locked away until the very end. Used *only once* to report the final Perplexity score.

Tuning hyperparameters on the Test Set is called "Data Leakage" and invalidates your research.

The Limits of Counting

Statistical LMs predict text by counting N-grams and calculating probabilities (MLE, Smoothing).

The Wall: N-gram models hit a hard ceiling.

1. **Context limitation:** Even a 5-gram only looks 4 words into the past. It cannot track long-term dependencies.
2. **Generalization failure:** It cannot share knowledge between similar words.

The New Era: **Deep Learning and Neural LMs abandon counting entirely.**

Words as Discrete Symbols

- **The Traditional View:** In statistics, words are atomic, discrete symbols.
- **Vocabulary (V):** A set of unique symbols: {cat, dog, car, apple...}
- **The Problem:** There is no inherent relationship between these symbols in a computer's memory.
- To a computer, the word cat is just as different from dog as it is from refrigerator.

One-Hot Encoding

- **How to feed words to a Neural Network?** Networks require numbers, not text.
- **One-Hot Encoding:** Representing a word as a massive vector.
- **The Architecture:** A vector of size $|V|$ (the entire vocabulary size).
- All elements are 0, except for a single 1 at the index corresponding to that specific word.
- We create a vector (a list of numbers) that is exactly as long as our entire vocabulary. If our vocabulary has 50,000 words, our vector has 50,000 slots.
- We put a 1 in the exact slot assigned to our word, and a 0 everywhere else.

The Curse of Dimensionality

- **The Engineering Nightmare:** Real-world vocabularies are massive ($|V| \approx 100,000+$).
- **Sparsity:** A one-hot vector is 99.999% zeroes.
- **Computational Inefficiency:** Storing and performing matrix multiplication on billions of zeroes wastes massive amounts of memory and processing power.
- **One-hot encoding scales terribly for modern NLP.**
- Imagine a real-world vocabulary of 100,000 words. To represent the single word "the", you have to store an array of 99,999 zeroes and a single 1. When a neural network processes a sentence, it has to do complex matrix multiplication on these gigantic, mostly empty arrays.

The Semantic Gap (Orthogonality)

- **The Mathematical Flaw:** One-hot vectors are orthogonal to each other.
- **Dot Product:** The mathematical measure of similarity between two vectors.

Let $\mathbf{v}_{\text{hotel}}$ and $\mathbf{v}_{\text{motel}}$ be one-hot vectors.

$$\mathbf{v}_{\text{hotel}} \cdot \mathbf{v}_{\text{motel}} = 0$$

- One-hot encoding mathematically guarantees that all words are completely dissimilar.

Distributional Semantics

Words that appear in similar contexts share similar meanings.

Example:

- I left my money at the **bank**.
- I left my money at the **institution**.
- (Therefore, **bank** and **institution** share semantic properties).

Because these words share the same surrounding context—the same "company"—they must share similar meanings. We can use this context to mathematically force similar words closer together.

Capturing Relational Meaning(Vector Arithmetic)

- The true magic of embeddings was revealed around 2013 by researchers at Google.
- Embeddings don't just capture similarity; they capture complex analogical relationships as geometric directions.
- **The Classic Equation:**

$$\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} + \mathbf{v}_{\text{Woman}} \approx \mathbf{v}_{\text{Queen}}$$

- Gender, verb tense, and geographical relationships are naturally encoded as mathematical offsets.
- The network organically learned the mathematical concept of gender offset simply by reading text! It works for countries and capitals, or present and past tense verbs too.

Standard Neural Networks

- **Feed-Forward Networks (FNNs):** Traditional neural networks process inputs in one direction, from input to output.
- **The Fatal Flaw for Language:**
 1. **Fixed Input Size:** FNNs require a fixed number of inputs (e.g., exactly 5 words). Language sequences are highly variable in length.
 2. **No Memory:** They process each input independently. If you feed in word 3, the network has completely forgotten word 2.
- **We need a network that can handle sequences of any length and maintain a "memory" of what it just read.**

Problem with Standard Neural Networks

Feed-Forward Networks (FNNs): Traditional neural networks process inputs in one direction, from input to output.

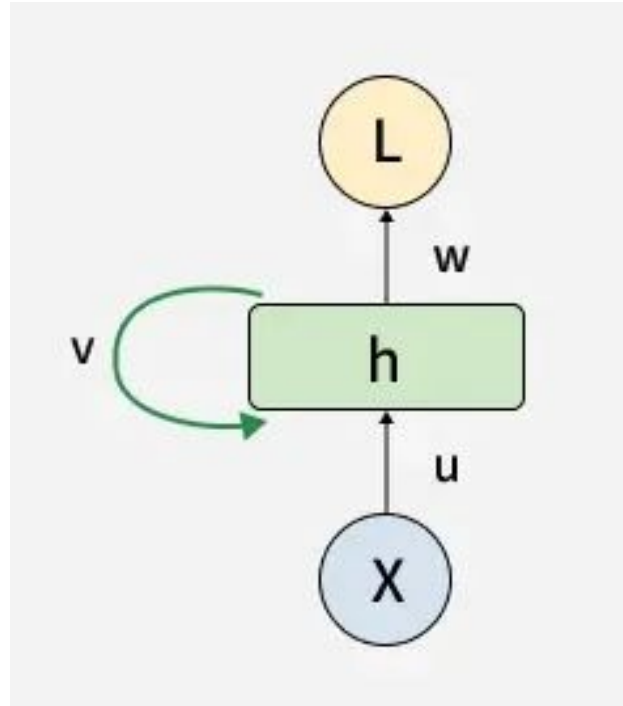
The Fatal Flaw for Language:

1. **Fixed Input Size:** FNNs require a fixed number of inputs (e.g., exactly 5 words). Language sequences are highly variable in length.
2. **No Memory:** They process each input independently. If you feed in word 3, the network has completely forgotten word 2.

Recurrent Neural Network (RNN)

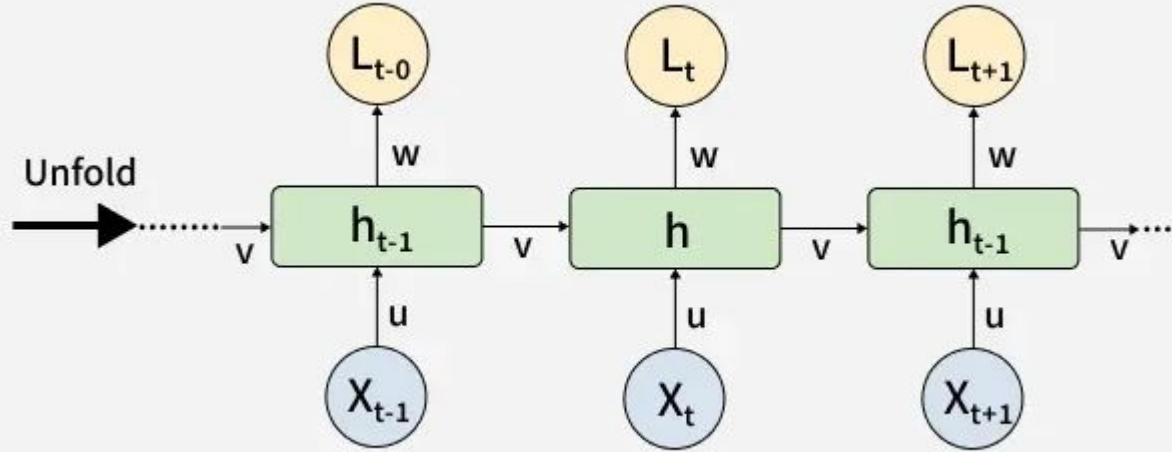
- **The Core Innovation:** A loop.
- **Recurrent Structure:** Instead of just passing information forward to the output, an RNN passes information *to itself* as it moves to the next step.
- **Time Steps (t):** The network processes the sequence one word at a time, where each word represents a distinct time step.

Recurrent Neural Network (RNN)



Recurrent units can "*remember*" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.

RNN Unfolding



Overall Function:

$$Y = f(X, h, W, U, V, B, C)$$

Hidden State Calculation:

$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

h represents the current hidden state.
 U and W are weight matrices.
 B is the bias.

. Output Calculation:

$$Y = O(V \cdot h + C)$$

O an activation function to the weighted hidden state where V and C represent weights and bias.

Updating the Hidden State in RNNs

1. State Update:

$$h_t = f(h_{t-1}, x_t)$$

where:

- h_t is the current state
- h_{t-1} is the previous state
- x_t is the input at the current time step

2. Activation Function Application:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

Here, W_{hh} is the weight matrix for the recurrent neuron and W_{xh} is the weight matrix for the input neuron.

3. Output Calculation:

$$y_t = W_{hy} \cdot h_t$$

where y_t is the output and W_{hy} is the weight at the output layer.

Recurrent Neural Network (RNN)

- **Recurrent Structure:** Instead of just passing information forward to the output, an RNN passes information *to itself* as it moves to the next step.
- **Time Steps (t):** The network processes the sequence one word at a time, where each word represents a distinct time step.
- It doesn't just push data up to the final output; it passes a copy of its current calculations back into itself to be used during the next step.
- We think of this processing in terms of "[time steps](#)."
- When reading the sentence "The cat sat," the word "The" is processed at time step $t=1$. The word "cat" is processed at $t=2$.

Types Of Recurrent Neural Networks

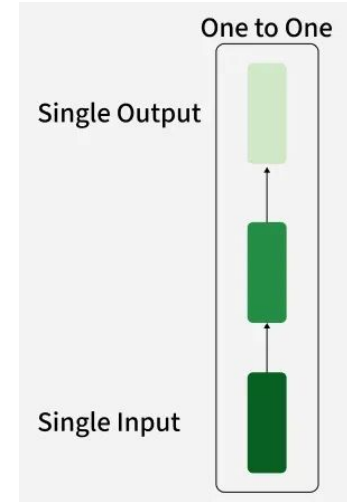
There are Four types of RNNs:

1. One - to - One RNN
2. One - to - Many RNN
3. Many - to - One RNN
4. Many - to - Many RNN

Types Of Recurrent Neural Networks

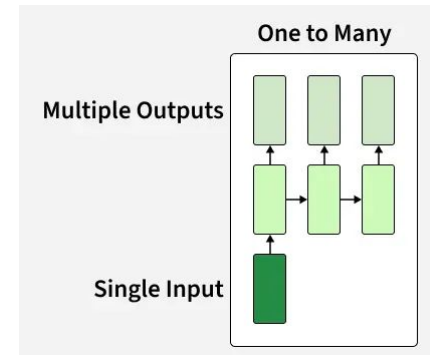
1. One - to - One RNN

- simplest type of neural network architecture
- There is a **single input and a single output**.
- It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



2. One - to - Many RNN

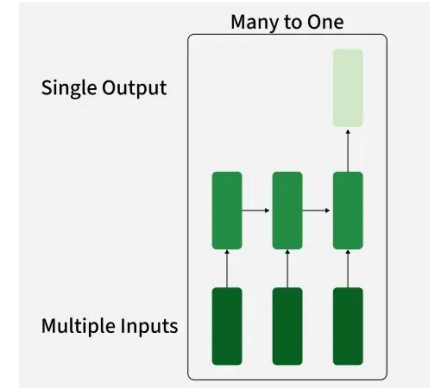
- single input to produce multiple outputs over time.
- This is useful in tasks where **one input triggers a sequence of predictions (outputs)**.
- For example in image captioning a single image can be used as input to generate a sequence of words as a caption.



Types Of Recurrent Neural Networks

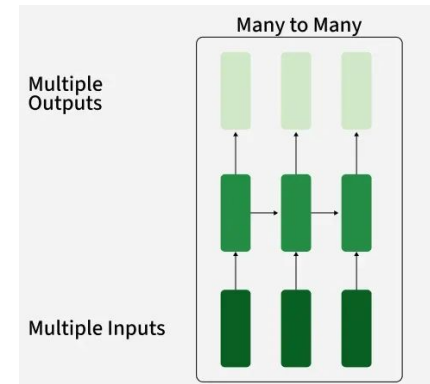
3. Many - to - One RNN

- Receives a sequence of inputs and generates a single output.
- Useful when the overall context of the input sequence is needed to make **one prediction**.
- In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



4. Many- to - Many RNN

- **sequence of inputs and generates a sequence of outputs.**
- In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.



Variants of Recurrent Neural Networks (RNNs)

1. Vanilla RNN

- simplest form of RNN - **single hidden layer** where weights are shared across time steps.
- Vanilla RNNs are suitable for learning short-term dependencies but are limited by the vanishing gradient problem, which hampers long-sequence learning.

2. Bidirectional RNNs

- **Process inputs in both forward and backward directions** (capturing both past & future context for each step.)
- This is ideal for tasks where the entire sequence is available, such as NER and question answering.

Variants of Recurrent Neural Networks (RNNs)

3. Long Short-Term Memory Networks (LSTMs)

- Introduce a **memory mechanism** to overcome the vanishing gradient problem.
- Each LSTM cell has three gates:
 - **Input Gate**: Controls how much new information should be added to the cell state.
 - **Forget Gate**: Decides what past information should be discarded.
 - **Output Gate**: Regulates what information should be output at the current step.
- This selective memory enables LSTMs to handle long-term dependencies, making them ideal for tasks where earlier context is critical.

4. Gated Recurrent Units (GRUs)

- Simplify LSTMs by combining the input and forget gates into a single update gate and streamlining the output mechanism.
- This design is computationally efficient, performing similarly to LSTMs, Useful in tasks where simplicity and faster training are beneficial.



NCMPE64 : Natural Language Processing and Generative AI

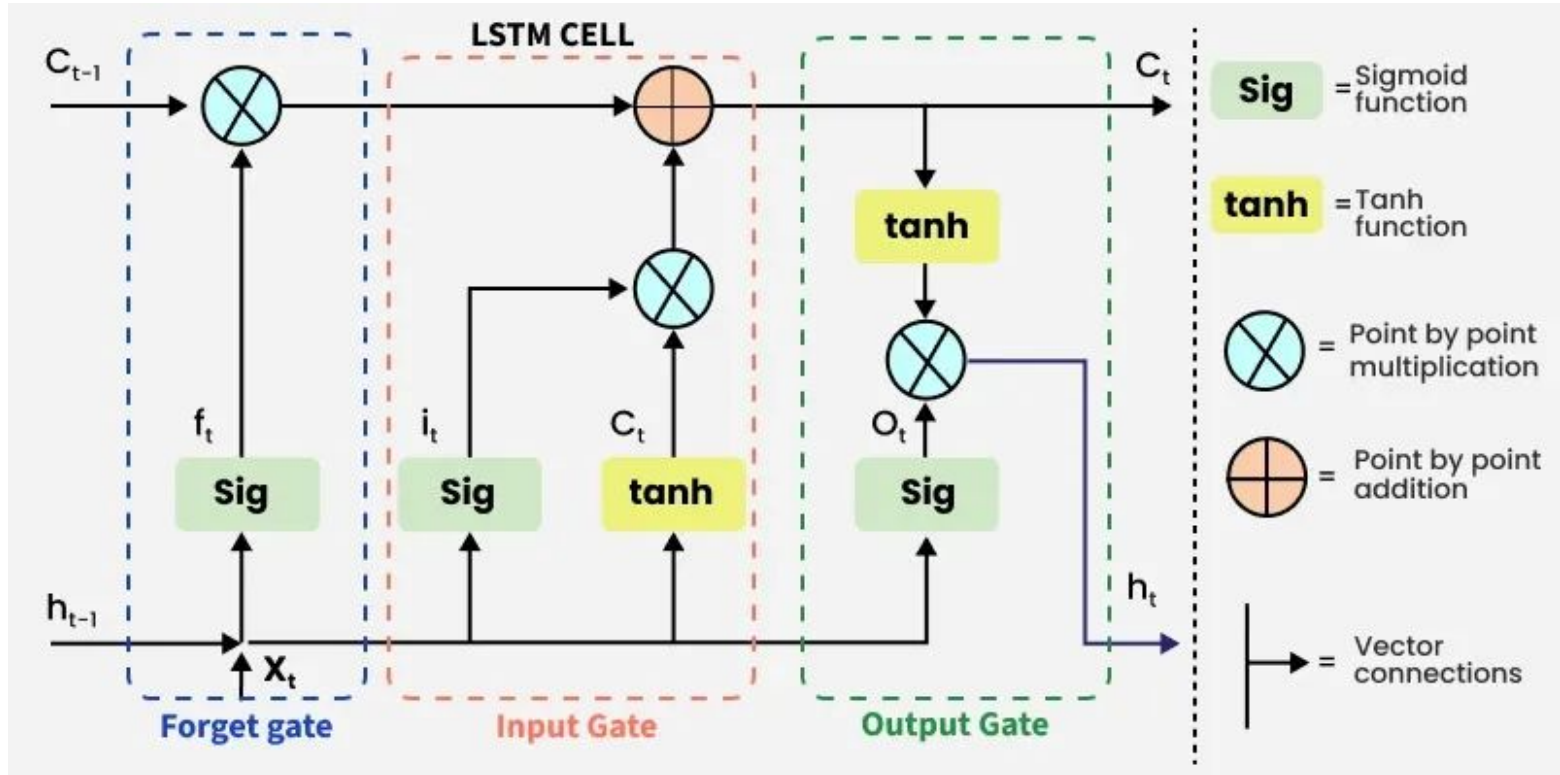
Long Short-Term Memory (LSTMs)

Solving Vanishing Gradients: LSTMs introduce a complex "cell state" that runs straight down the entire chain, acting like a conveyor belt for memory.

The Three Gates: LSTMs control information flow using specialized neural layers called gates:

- **Forget Gate:** Decides what information to throw away from the past state.
- **Input Gate:** Decides what new information to add to the cell state.
- **Output Gate:** Decides what parts of the cell state to output as the new hidden state

Long Short-Term Memory (LSTMs)





NCMPE64 : Natural Language Processing and Generative AI